

Global-View Reasoning with LLMs for Real-World Vulnerability Detection

Xiaozhen Xu*
24210240359@m.fudan.edu.cn
Fudan University
Shanghai, China

Chaoran Tao*
crtao26@m.fudan.edu.cn
Fudan University
Shanghai, China

Guangliang Yang†
yanggl@m.fudan.edu.cn
Fudan University
Shanghai, China

Xinming Guo
25213050169@m.fudan.edu.cn
Fudan University
Shanghai, China

Can Li
25213050216@m.fudan.edu.cn
Fudan University
Shanghai, China

Yangshuo Bai
25213050100@m.fudan.edu.cn
Fudan University
Shanghai, China

Yi Wang
25213050388@m.fudan.edu.cn
Fudan University
Shanghai, China

Abstract

Large language models (LLMs) have recently shown promise in code generation and other software engineering tasks. However, their effectiveness in real-world security vulnerability detection still remains limited. A key observation is that while LLMs exhibit strong semantic understanding and security analysis capabilities on code snippets, these strengths are rarely extended to large-scale real-world programs. In this work, we propose SemScan, a novel vulnerability detection approach that harnesses LLMs' local reasoning capabilities to enable global security analysis. SemScan establishes an iterative loop of evidence collection and evidence-guided semantic synthesis. Our evaluation on a Python CVE vulnerability dataset and 200 popular real-world Python projects demonstrates that SemScan significantly outperforms state-of-the-art tools, uncovering 62 previously unknown serious vulnerabilities, including remote code execution and insecure deserialization flaws. To date, 22 of these vulnerabilities have been confirmed and assigned CVE identifiers.

CCS Concepts

• Security and privacy → Software security engineering.

Keywords

Vulnerability Detection; Static Analysis; Large Language Model based Analysis; Software Security

ACM Reference Format:

Xiaozhen Xu, Chaoran Tao, Guangliang Yang, Xinming Guo, Can Li, Yangshuo Bai, and Yi Wang. 2026. Global-View Reasoning with LLMs for Real-World Vulnerability Detection. In *Proceedings of the 2026 ACM SIGSAC*

*Both authors contributed equally to this work.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCS '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2871-6/2026/11

<https://doi.org/10.1145/3830454.3846788>

Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages.
<https://doi.org/10.1145/3830454.3846788>

1 Introduction

Large Language Models (LLMs) have made significant progress in recent years and demonstrated powerful capabilities in natural language understanding, logical reasoning, complex problem solving, code generation, and other software engineering tasks [15, 22, 29, 40]. They have been widely adopted across a broad range of critical domains, including autonomous systems, healthcare diagnostics, content creation, and scientific discovery [11, 25, 35, 44]. This widespread adoption marks a pivotal shift that LLMs have moved beyond experimental prototypes and are now becoming integral and general-purpose components of modern software systems.

In the context of software security, LLMs have demonstrated early yet promising potential. Prior work has explored their use in assisting a variety of security tasks, including recognition of insecure code patterns [2], reduction of false positives in static analysis results [5], enhancement of dynamic testing [31], and even automated vulnerability patch generation [37]. LLMs exhibit a strong capacity to infer program semantics (e.g., taint flows) and assess their security properties. More recently, LLM systems have increasingly adopted agentic designs that integrate retrieval and external tools. For example, retrieval-augmented approaches such as Vul-RAG [7] retrieve relevant vulnerability knowledge from external databases to assist in determining whether a given code snippet is vulnerable.

Challenges. However, these successes are largely confined to small-scale or localized code scenarios. Extending LLMs to real-world large-scale programs remains fundamentally challenging, particularly when LLMs are expected to lead rather than only assist the analysis process. Recent empirical studies corroborate this limitation, showing that the performance of LLM-based approaches

degrades substantially on real-world projects [19]. These limitations stem from three fundamental constraints.

First, there is an inherent mismatch between the bounded context window of LLMs and the need for holistic program understanding. To mitigate this, prior approaches such as Magneto [43], LLMxCPG [17], and RepoAudit [12] adopt decomposition-based designs that partition programs into smaller units and apply LLMs locally. While this improves tractability, it inevitably fragments program semantics and breaks cross-component dependencies, making it difficult to capture vulnerabilities that span multiple scopes. IRIS [20] faces a similar constraint. LLMs only plays a subordinate role, assisting an underlying static analysis engine that still inherits the same bounded, decomposition-based view of the program.

Second, LLMs struggle to sustain coherent reasoning over long and interdependent execution paths [32, 33]. Real-world vulnerabilities often emerge from chains of operations across functions, files, and components, requiring consistent propagation of intermediate analysis results. Nonetheless, as reasoning depth increases, LLMs tend to lose coherence, leading to forgotten assumptions and contradictory intermediate conclusions. Consequently, related tools such as RepoAudit restrict analysis to shallow method-level reasoning with bounded depth (i.e., up to four call sites), thereby failing to capture long-chain execution paths. Similarly, Vul-RAG and Lift [18] only perform single-shot reasoning on code snippets or reports.

Last, LLMs exhibit unstable and unreliable reasoning in large and noisy programs. Vulnerability-relevant signals are typically sparse and deeply buried, yet LLMs lack robust mechanisms to distinguish critical semantics from irrelevant context. As a result, they are prone to hallucinating infeasible execution paths or spurious taint-flow connections, significantly undermining analysis reliability [13, 38].

Our Approach. In this paper, we propose SemScan, a novel LLM-based agentic vulnerability detection approach for real-world large-scale projects. The key idea behind SemScan is to iteratively assemble a coherent global view from fragmented and evidence-backed observations. Specifically, localized observations (such as a variable assignment, a function call, or an object field update) provide partial semantic evidence about program behavior. In isolation, such evidence cannot reveal how untrusted input propagates to security-sensitive operations. By systematically synthesizing these distributed evidence fragments, it becomes possible to reconstruct complete source-to-sink relationships. From this perspective, vulnerability detection can be viewed as a problem of evidence-grounded semantic synthesis, rather than isolated pattern recognition.

Following the idea of global-view semantic synthesis, our SemScan approach proceeds as follows. First, SemScan processes the target program to construct a coarse-grained structural and semantic context as initial evidence. Then, based on the accumulated evidence, SemScan conducts semantic reasoning and synthesis to construct candidate source-to-sink taint flows. Since the current evidence set is often insufficient to support complete taint flow construction, an evidence induction technique is applied to generate targeted evidence collection tasks for semantic exploration. Last, SemScan leverages a tool execution pipeline to explore and

collect new evidence using a rich set of tools that support repository indexing and static analysis.

This whole process forms an iterative loop. Each cycle accumulates additional evidence, while the synthesis engine re-derives candidate taint flows from scratch over the updated evidence set, preventing earlier reasoning errors from being carried over. Such global-view semantic analysis captures how program components interact across multiple levels of abstraction and enables accurate identification of potential vulnerabilities. Moreover, SemScan operates directly on source code. Once user-specified sources and sinks are provided, it automatically reconstructs candidate end-to-end taint flows grounded in verifiable evidence. To optimize inference cost, SemScan employs a heterogeneous mixture of LLMs, significantly reducing inference cost without compromising detection effectiveness.

Our SemScan approach effectively addresses the three key obstacles. First, evidence induction incrementally accumulates security-relevant evidence to construct a global security view within a bounded LLM context. Second, the re-synthesis sustains correctness over long inter-procedural chains, preventing early errors from compounding as reasoning depth grows. Last, every accepted taint step must be grounded in concrete evidence, mitigating hallucinated propagation.

Evaluation. We implement a prototype of SemScan for Python, a widely used dynamically typed language whose language features make large-scale vulnerability detection particularly challenging. We systematically evaluate SemScan from several perspectives. First, we assess its effectiveness on a benchmark of 30 real-world Python CVE vulnerabilities spanning five distinct categories. SemScan successfully detects 28 cases (93.33%). We compare SemScan with five baselines: CodeQL [10] and Semgrep [30] as industrial static analyzers, GPT-5.4+ManualCraft as an LLM baseline, RepoAudit and Claude Code [1] as agent-based LLM approaches. CodeQL and Semgrep achieve detection rates of 3.33% and 0%, respectively, while GPT-5.4+ManualCraft achieves 63.33%. RepoAudit and Claude Code achieve 20% and 70%, respectively. Second, we apply SemScan to 200 popular real-world large-scale Python projects to evaluate its effectiveness in large-scale settings. SemScan uncovers 62 previously-unknown (manually confirmed, 0-day) vulnerabilities across 39 projects, including severe issues such as RCE and insecure deserialization. On average, each vulnerability analysis takes 6.2 minutes and costs \$0.065. This efficiency comes from a decomposed architecture that uses heterogeneous LLMs, making SemScan both effective and economically viable for real-world deployment.

We have conducted responsible disclosure of all our findings to the developers and are actively helping them to fix the identified vulnerabilities. To date, 22 vulnerabilities have been assigned CVE identifiers.

In summary, this paper has made the following contributions:

- We rethink taint-style vulnerability detection as an evidence grounded semantic synthesis problem, where complete source-to-sink paths are derived from incomplete but verifiable local program evidence.

- We design evidence induction with iterative evidence-grounded re-synthesis, which turns missing semantic links into targeted evidence-collection tasks and reconstructs taint paths over accumulated evidence.
- We present and implement SemScan, a source-level vulnerability detection system that integrates LLM-based local reasoning with a global security view, enabling end-to-end automated analysis given a repository path and sources and sinks.
- By applying SemScan to popular large-scale real-world Python projects, we identify 62 confirmed previously unknown vulnerabilities. Up to now, 22 of these vulnerabilities have been assigned CVE identifiers.

2 Background and Motivation

2.1 Background

As the size and structural complexity of real-world codebases have increased substantially over the past decades, the number of publicly disclosed software vulnerabilities has continued to grow [14, 36]. Comprehensive manual security analysis is difficult to conduct in practice. Consequently, automated vulnerability detection techniques have become an indispensable component of modern software security engineering. Static program analysis based approaches [8, 10] can be widely used. They can extract program semantics (e.g., control and data flows) and vet their security without actually executing target programs.

Deep learning based approaches [21, 26, 42] have been proposed for vulnerability detection. By learning from labeled vulnerable and non-vulnerable code, these models can capture statistical patterns associated with security flaws without requiring manually defined rules. However, they typically rely on localized code and lack explicit semantic reasoning, limiting their effectiveness in real-world programs.

Recently, LLMs have been increasingly explored for their utility in software security. They have demonstrated potential in assisting specific tasks, such as interpreting code semantics and identifying local insecurity patterns [32, 41]. Nevertheless, current LLM-based approaches are constrained by a ‘local view’ problem. Due to fixed context windows, they typically face difficulties of effectively analyzing real-world vulnerabilities which often span over multiple functions and files.

2.2 Motivating Example

In this section, we use a real-world CVE vulnerability to illustrate the challenges of detecting vulnerabilities in real-world large-scale software. The vulnerability CVE-2026-24123 [27] is a path traversal flaw in the high-profile artificial intelligence (AI) model-serving framework ‘BentoML’ (8.6K stars in GitHub).

Figure 1 shows the simplified relevant code. The vulnerability originates from a user-controlled configuration file `bentofile` passed to the entry point `build_bentofile()` (Line 1). When `bentofile` is provided, the program invokes `BentoBuildConfig.from_file()` (Line 3) to parse it into a configuration object `build_config`. The parsing process further reaches `BentoBuildConfig.load()`, where the third-party function `bentoml_catr.structure()`

is called (Line 34). This function performs schema-driven deserialization according to the runtime class `cls`. Specifically, the user-controlled entry `data["docker"]` is matched against the `docker` field declared in `BentoBuildConfig` (Lines 23–25) and is used to construct the corresponding `DockerOptions` object. Since this assignment is performed inside the third-party library, it does not appear as an explicit parameter passing or field assignment in the project code, but it implicitly embeds attacker-controlled data into the object state, i.e., `build_config.docker`.

The tainted configuration object is then passed to `Bento.create()` (Line 7). Inside this method, the persisted field `build_config.docker` is dereferenced and used to invoke `write_to_bento()` (Line 17). This call transfers the tainted object state to the receiver object of `DockerOptions.write_to_bento()`. The critical sink occurs in this method: `self.setup_script`, which originates from the earlier deserialized configuration, is accessed (Line 38) and passed to `shutil.copy()` as the source path (Line 39). Consequently, an attacker can craft the `setup_script` value in the configuration file to copy unintended files into the generated Bento artifact, resulting in a path traversal vulnerability.

Why Vulnerability Detection Is Difficult. Detecting this vulnerability is challenging due to two main obstacles:

❶ *Library-driven implicit field assignment at Line 34:* The third-party library API `bentoml_catr.structure()` internally assigns untrusted input from `data["docker"]` to `build_config.docker`. This assignment follows the API’s runtime schemas. There is neither explicit parameter passing nor direct field writing in the source code. Therefore, static analyzers often treat such third-party library calls as black boxes, causing taint propagation to be interrupted at the library boundary and preventing subsequent object state changes from being tracked. RepoAudit, in contrast, uses LLMs only as local code interpreters, and queries cross-function and cross-component dependencies through an AST-based call graph. This approach remains subject to the common limitations of traditional static analyzers. Likewise, it cannot model the runtime schema-driven dispatch logic inside `bentoml_catr.structure()`, causing its analysis exploration to terminate once it reaches the library boundary.

❷ *Persistent cross-layer field access via implicit receivers at Lines 17 & 38:* After deserialization, tainted data persists in heap-allocated object fields and is accessed across multiple abstraction layers. Specifically, the field `build_config.docker` is dereferenced at Line 17, and the tainted state further propagates to `self.setup_script`, which is accessed at Line 38 inside a nested method call. Such cross-layer accesses through implicit receivers make it difficult for static analysis to maintain precise field- and alias-sensitive tracking, leading to missed vulnerabilities. For RepoAudit, since propagation already terminates at the library boundary in the first challenge, the subsequent cross-layer accesses at Lines 17 and 38 are never reached.

Together, these factors expose fundamental limitations of both traditional static analyzers and LLM-based approaches that rely on local, decomposition-based reasoning. Static analyzers cannot



Figure 1: Motivating Example

model runtime schema-driven dispatch or maintain precise field-sensitive tracking across abstraction layers. RepoAudit, representative of such decomposition-based designs, cannot model the runtime schema-driven dispatch inside third-party library calls, causing it to silently terminate at the library boundary and never reach the subsequent cross-layer accesses. When vulnerabilities rely on library-driven implicit assignments and persistent cross-layer object fields, tools lacking a global semantic view cannot maintain a continuous, precise data-flow. This motivates SemScan, which synthesizes semantic links across fragmented, evidence-backed observations to recover complete taint flows.

2.3 Threat Model

We consider real-world programs deployed in practical environments as the target applications. We assume that the target program is developed and maintained by benign developers, and that the attacker does not initially compromise the host environment or modify the program code. We focus on taint-style vulnerabilities, where security violations arise from unsafe flows of attacker-controlled data from externally controllable sources to security-sensitive sinks, such as SQL injection, OS command injection, code injection, and path traversal. Accordingly, the attacker is modeled as a malicious user who can interact with the program through normal input interfaces, such as network requests, configuration files, command-line arguments, or user-provided data files. By crafting malicious inputs, the attacker aims to influence data flows that propagate from externally controllable sources to security-sensitive sinks, thereby enabling attacks such as remote code execution or sensitive data leakage.

3 SemScan Design and Implementation

In this paper, we introduce SemScan, a novel LLM-driven security vetting framework for real-world large-scale Python programs. This section breaks down the design and implementation of SemScan.

3.1 Research Goal

In this work, we target taint-style vulnerabilities in real-world Python programs, where the central challenge is to accurately reconstruct taint paths. Such paths capture how potentially malicious input can propagate from program entry points and reach sinks where it may cause security violations.

Assume that $C = \{c_0, c_1, \dots\}$ is a set of source points, and $K = \{k_0, k_1, \dots\}$ is a set of sink points. Each source c represents a program location where external input enters the system, and each sink k represents a security-critical operation, such as a database query and command execution. Each taint path p_c^k is an ordered sequence of data-flow steps T , i.e., $p_c^k = \{c, t_0, t_1, \dots, k\}$ (where $t \in T$), that links the source c and the sink k . Each step t corresponds to one concrete program operation (e.g., assignment) contributing to advancing taint propagation. The objective is to construct a set of taint paths P , i.e., $p \in P$.

3.2 Key Idea of Evidence Induction

A fundamental obstacle to achieving the research goal defined in Section 3.1 lies in reasoning about the taint path p_c^k between a given source c and sink k in the target large-scale program. Our key idea is to formulate taint path construction as an evidence-grounded semantic synthesis problem. We treat program semantics (e.g., data flows) as analysis facts, i.e., *semantic evidence*. In general, at the beginning of the analysis, as only the pair (c, k) is explicitly available, the semantic evidence is inherently incomplete. SemScan can incrementally expand the evidence and synthesize taint paths over the accumulated evidence. A taint path is identified and accepted when it is supported by concrete traceable evidence.

It is worth noting that the evidence E consists of one or more localized data-flow steps, i.e., $\{t_i, t_{i+1}, \dots, t_{i+n}\}$, each capturing a partial taint propagation relation. E establishes the ground of the synthesis of the potential taint path.

3.2.1 Evidence Induction. It is critical to collect and expand evidence. We formulate evidence construction as an induction process, referred to as evidence induction. Starting from an initial evidence set, the analysis acquires new evidence and progressively

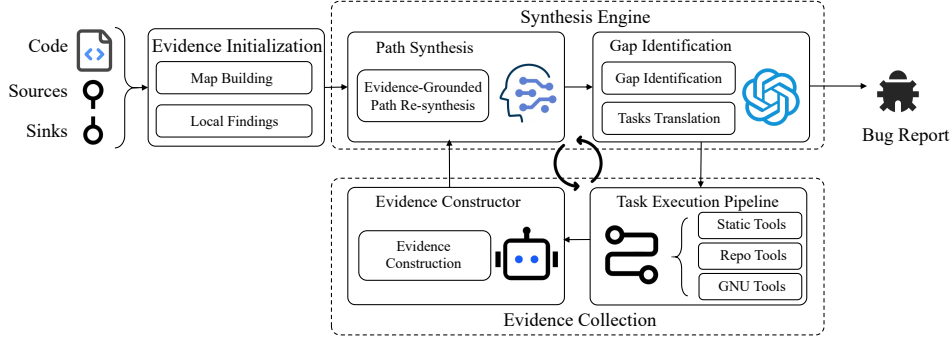


Figure 2: SemScan Workflow

accumulates the evidence set until a taint path can be reconstructed or no further evidence can be obtained.

Base Case. Given a source-sink pair (c, k) , the initial accumulated evidence set E_0 is constructed from two complementary components:

$$E_0 = E_{\text{local}} \cup E_{\text{struct}}.$$

Here, E_{local} captures localized evidence around c and k , such as how external inputs enter the program and where security-sensitive operations occur. E_{struct} represents structural evidence, including module dependencies and class hierarchies.

Inductive Step. At iteration j , let E_j denote the current accumulated evidence set. Based on E_j , we acquired incremental evidence as ΔE_j , which establishes semantic links that have control-flow or data-flow relations with facts in E_j . The accumulated evidence set E_{j+1} is then updated monotonically:

$$E_{j+1} = E_j \cup \Delta E_j.$$

3.2.2 Applying Evidence Induction For Taint Path Synthesis. Following the evidence induction process, one key step at each iteration is to derive E_{j+1} from the current evidence set E_j . To achieve this, we can leverage LLM to perform goal-guided analysis. Specifically, the LLM can first attempt to synthesize a candidate taint path from the available evidence E_j . When the synthesized taint path is incomplete, it indicates that supporting evidence is missing. These missing evidence is identified as evidence gaps with respect to the target path. Then, we can translate the identified gaps into targeted evidence collection tasks. Through LLM reasoning and corresponding tool actions (e.g., code indexing and static analysis), these tasks may resolve implicit control transfers (e.g., indirect calls and callbacks) and construct data-flow relations related to E_j , thus discovering new evidence. The newly discovered evidence is denoted as ΔE_j and incorporated to E_{j+1} . E_{j+1} is employed to conduct the next round of analysis. This process inductively enlarges the accumulated evidence set until a taint path can be reconstructed or no further evidence can be obtained.

3.2.3 Induction Termination. The induction process is bounded by two properties. First, the accumulated evidence set form a monotonic sequence: $\{E_0 \subseteq E_1 \subseteq E_2 \subseteq \dots\}$. This is because each iteration only adds newly collected evidence ΔE_j and never discards previously accepted evidence. Second, the space of semantic

facts in the target program is finite, which bounds the total amount of evidence that can be accumulated. In practice, the process terminates when either: (1) $E_j \Rightarrow p$, meaning that the current accumulated evidence set is sufficient to synthesize a taint path p ; or (2) $\Delta E_j = \emptyset$, indicating that the current iteration cannot acquire additional grounded evidence to close the remaining gaps. In the latter case, the evidence set reaches a stable state, i.e., $E_{j+1} = E_j$, and no further evidence expansion is performed.

3.3 Main Workflow

As shown in Figure 2, SemScan follows an evidence-grounded workflow to reconstruct source-to-sink taint paths. This workflow is built upon evidence induction, which progressively integrates fragmented local evidence into a coherent global security view of the program.

(1) Evidence Set Initialization. SemScan first processes the target program to initialize the evidence set. The initialized evidence set provides the base evidence E_0 for the first synthesis round.

(2) Path Synthesis. Given an evidence set E_j , SemScan performs evidence-grounded semantic synthesis to construct taint paths. In the first round, synthesis is performed over the initialized evidence set; in later rounds, newly constructed evidence is incorporated into the evidence set to produce an updated E_{j+1} , and SemScan re-synthesizes taint paths from scratch over the accumulated evidence.

(3) Gap Identification. Based on the current synthesis result, SemScan identifies missing evidence gaps, where each gap corresponds to a missing evidence. These gaps are translated into concrete evidence-collection tasks.

(4) Evidence Collection. For each task, SemScan invokes a task-specific execution pipeline to retrieve the required evidence, leveraging various tools. SemScan employs an auxiliary lightweight LLM to normalize and refine tool results into a unified evidence representation. The collected evidence is then returned to the synthesis step, where it is incorporated into the accumulated evidence set for the next round of path re-synthesis.

This workflow is guided by four design principles that explain how SemScan scales evidence-grounded reasoning to large real-world programs.

Principle#1: Evidence induction supports bounded LLM inputs. Instead of analyzing the entire program monolithically, SemScan performs evidence induction driven by evidence gaps. Each gap specifies a missing evidence, enabling SemScan to focus only on the program regions necessary to acquire that evidence. This design allows SemScan to scale to large program while keeping each model input bounded and relevant.

Principle#2: Re-synthesis over accumulated evidence stabilizes long-chain reasoning. Incrementally extending a prefix can propagate early errors through later reasoning steps. As new evidence becomes available, SemScan re-synthesizes the full taint-path set from scratch, allowing paths to be revised and expanded. This enables later evidence to correct earlier incomplete interpretations and improves reasoning over long cross-file and inter-procedural flows.

Principle#3: Evidence-grounded synthesis mitigates hallucination. LLM-based reasoning may produce infeasible taint paths due to hallucination. SemScan addresses this by enforcing strict evidence grounding. A taint path is considered valid only if every step is supported by concrete evidence in the accumulated evidence set. This constraint improves the reliability of the analysis.

Principle#4: Decomposed execution reduces analysis cost. To reduce cost, SemScan separates local evidence construction from global path synthesis. The key is that not all analysis steps require the same reasoning capability. Many local steps only require extracting data-flow semantics from bounded code fragments, which can be reliably handled by small-scale and lightweight LLMs. Therefore, SemScan decomposes the workload by reasoning complexity. The synthesis engine utilizes the large-scale and high-performance LLM for complex global reasoning over the accumulated evidence, while the evidence constructors use the lightweight LLMs to handle simpler local tasks. The task execution pipeline further avoids unnecessary LLM invocations by executing tools, including code retrieval, static navigation, and pattern search.

3.4 Evidence Set Initialization

SemScan performs initialization to produce coarse-grained traceable base evidence for the first analysis round of the synthesis engine (i.e., base case). Specifically, we construct a repository map capturing structural evidence, including module dependencies and boundaries. For each source and sink, we generate localized evidence including:

- The relevant code snippet around the location.
- File path and start/end line numbers of the above code snippet.
- A concise summary describing local data-flow behavior within the snippet, generated by an evidence constructor.

This module provides an intentionally incomplete global view. It offers just enough structural and semantic context to guide subsequent reasoning, while deferring detailed exploration to inductive evidence expansion.

3.5 Evidence-Grounded Synthesis Engine

SemScan orchestrates multi-round global reasoning and synthesis over the accumulated evidence. As stated in Section 3.2.2, SemScan synthesizes source-to-sink taint paths over the current evidence set E_j . An evidence gap is recorded when the available evidence is insufficient to synthesize and infer a taint path. This gap guides evidence induction.

3.5.1 Gap-guided Evidence Expansion. Evidence expansion in SemScan proceeds inductively guided by evidence gaps. Each gap characterizes what evidence is missing, such as an unresolved callee, a missing alias relation, an implicit field update, or a library-mediated object construction. Based on the gap, SemScan generates targeted collection tasks to acquire the missing evidence. These tasks are executed through the task execution pipeline and summarized by evidence constructors into new evidence i.e., ΔE_j .

Concrete example (gap $\rightarrow$ plan step $\rightarrow$ evidence). Consider a synthesis step where the engine needs to understand how a model parameter in `LLM.__init__` propagates to `EngineArgs`, but the relevant code is not yet in E_j . The engine records this as an unresolved gap:

```
1 Gaps: ["how the model parameter in LLM.__init__ is passed to
        EngineArgs"]
```

A targeted plan step is generated to locate the enclosing method:

```
1 [{
2   "id": "s0",
3   "tool": "static.locate_method_by_line",
4   "args": {
5     "rel_path": "vllm/entrypoints/llm.py",
6     "line": 187}
7 }]
```

The pipeline executes the step and returns a bounded navigation result, which evidence constructors summarize into new evidence, closing the gap and extending the global view.

3.5.2 Iterative Path Re-Synthesis for Multi-Step Reasoning. After new evidence is constructed, the engine performs a fresh end-to-end synthesis over the updated evidence set E_{j+1} . Each synthesis round attempts to infer taint paths from the current evidence set and retains only the paths whose data-flow steps are explicitly grounded in supporting evidence. This evidence-grounded constraint ensures that path construction remains closely aligned with observed code behavior. Hence, hallucinations are mitigated by preventing the engine from freely inferring unsupported propagation.

Rather than extending partial paths from the previous round, SemScan re-synthesizes taint paths after each evidence update. Paths synthesized under insufficient evidence may contain incorrect propagation or miss critical propagation branches, and directly extending them can amplify these errors in later reasoning steps. Re-synthesis allows newly collected evidence to reshape the path structure and revise earlier decisions, thereby reducing error propagation in long-chain reasoning.

The iterative loop terminates when SemScan synthesizes complete evidence-backed source-to-sink paths, or when no more evidence can be accumulated. At termination, the current candidates represent the taint paths that can be supported under the accumulated evidence.

3.6 Task-Execution Pipeline

To ensure controlled and scalable gap resolution, SemScan relies on a task execution pipeline that mediates all interactions between the synthesis engine, evidence constructors, and the target repository. Neither the engine nor the evidence constructors access source code directly. All code retrieval, inspection, and analysis actions are performed through tool invocations executed by the pipeline. This separation enforces a clear boundary between *reasoning* and *execution*, guaranteeing that all evidence used for synthesis is traceable to concrete code artifacts.

The pipeline exposes a set of constrained analysis tools, which can be broadly categorized into three classes:

(1)**Repository-indexing tools** offer controlled access to project structure and source files, including repository tree inspection, directory listing, bounded file reading, and text search. These tools allow incremental exploration of large repositories without exposing excessive or irrelevant code context.

(2)**Static-analysis tools** perform lightweight language-specific analyses, such as locating function or class definitions and determining the enclosing scope of a given line via abstract syntax trees. Outputs from static tools are treated as navigation hints; evidence must always be obtained by retrieving the corresponding code through repository tools.

(3)**Pattern-search tools** (e.g., regex-based code search) support efficient, bounded exploration of large codebases for specific patterns or structures, enabling scalable identification of code regions.

By restricting all interactions to a predefined set of tools, SemScan enforces a controlled execution interface between the synthesis engine, evidence constructors, and the repository. This design ensures that all code access and evidence retrieval are explicitly mediated, avoiding uncontrolled context growth.

3.7 Evidence Constructor

After gaps are resolved, evidence constructors are responsible for collecting tool outputs and normalizing the results into structured, traceable evidence objects that serve as the input for the synthesis engine. Each evidence object captures the local data-flow semantics of a bounded code fragment, together with its location in the program. To improve performance, evidence constructors run in parallel, each independently constructing evidence objects from their assigned code fragments.

3.7.1 Evidence Construction. Evidence constructors are designed as lightweight LLMs that collect and process outputs from the task execution pipeline, which provides relevant code fragments retrieved from the program.

Upon receiving a fragment, a constructor (as a lightweight LLM) generates a concise summary that abstracts essential semantic information, without performing global reasoning. The summary is then combined with the original fragment into a normalized evidence object, anchored with file paths and line ranges.

The evidence objects are composable. The synthesis engine integrates them across files and modules, resolving symbols and connecting inter-procedural data flows, while keeping reasoning compact and minimizing hallucinations. This enables the synthesis engine to reconstruct reliable taint paths, even in large and complex repositories.

3.7.2 Evidence Schema and Concrete Example. Each evidence unit is formally represented as a tuple:

(summary, file_path, start_line, end_line, raw_code)

where:

(1) **summary**: a concise description of the local data-flow semantics within the snippet;

(2) **file_path**: the repository-relative path to the source file containing the snippet;

(3) **start_line / end_line**: line range spanned by the snippet;

(4) **raw_code**: the actual code extracted from the repository.

This schema is intentionally minimal. The summary captures the localized semantics needed for subsequent composition by the synthesis engine, while the file path and line range ensure traceability and allow evidence to be reliably integrated across multiple snippets and files. Minimizing additional metadata keeps the reasoning context compact, reducing the overhead for the synthesis engine.

For example, consider the following Python snippet where user input is propagated to a system command:

```
1 def execute_user_command():
2     user_input = input('Enter command: ')
3     os.system(user_input)
```

The evidence constructor would generate the following evidence object:

```
1 {
2     "summary": "Input from input() is sent to os.system()",
3     "file_path": "app/utills/execution.py",
4     "start_line": 10,
5     "end_line": 12,
6     "raw_code": "10: def execute_user_command():\n11: \
7     tuser_input = input('Enter command: ')\n12: \t os.\n    system(user_input)\n"
```

4 Experiments

Our evaluation seeks to answer the following five research questions:

- **RQ1:** How effective is SemScan in identifying complex vulnerabilities within a controlled benchmark compared to state-of-the-art tools?
- **RQ2:** How do the choices of synthesis engine and the parameter scale of evidence constructors impact the detection performance?
- **RQ3:** To what extent do the gap-guided Expansion and iterative re-synthesis mechanisms improve the system's performance?
- **RQ4:** Can SemScan discover previously unknown vulnerabilities in large-scale, high-profile open-source projects?
- **RQ5:** What are the computational costs and latencies associated with SemScan's decomposed architecture?

4.1 Experimental Setup

Prototype Implementation. We implement the prototype of our SemScan approach. The prototype contains 1692 lines of Python code, including 684 lines for the task execution pipeline and tools,

Table 1: Benchmark dataset and detection results.

Category	CVE ID	CWE	Project	Depth	Detection Result					
					CodeQL	Semgrep	RepoAudit	GPT-5.4+ManualCraft	Claude Code	SemScan
Code Injection	CVE-2024-1881	CWE-78	AutoGPT	6				✓	✓	✓
	CVE-2024-3271	CWE-77	LlamaIndex	5				✓	✓	✓
	CVE-2025-25362	CWE-94	Spacy-LLM	2				✓	✓	✓
	CVE-2026-0761	CWE-94	MetaGPT	5				✓	✓	✓
	CVE-2026-0766	CWE-94	Open WebUI	3	✓			✓	✓	✓
	CVE-2026-22807	CWE-94	vLLM	9				✓	✓	✓
Data Injection	CVE-2023-29374	CWE-74	LangChain	3				✓	✓	✓
	CVE-2024-6331	CWE-74	devika	9			✓	✓	✓	✓
	CVE-2024-11958	CWE-89	LlamaIndex	3			✓	✓	✓	✓
	CVE-2024-12909	CWE-89	LlamaIndex	3				✓	✓	✓
	CVE-2024-23731	CWE-88	Mem0	10				✓	✓	✓
File/Path Vulnerabilities	CVE-2023-51449	CWE-22	Gradio	3			✓	✓	✓	✓
	CVE-2024-4941	CWE-22	Gradio	9				✓	✓	✓
	CVE-2024-8953	CWE-627	composio	8				✓	✓	✓
	CVE-2024-8958	CWE-434	Composio	8				✓	✓	✓
	CVE-2024-11042	CWE-73	invokeai	6			✓	✓	✓	✓
	CVE-2024-28088	CWE-22	LangChain	8				✓	✓	✓
	CVE-2026-24123	CWE-22	BentoML	10				✓	✓	✓
Deserialization and Template Execution	CVE-2023-6730	CWE-502	Transformers	4				✓	✓	✓
	CVE-2023-7018	CWE-502	Transformers	8				✓	✓	✓
	CVE-2024-9053	CWE-502	vLLM	4			✓	✓	✓	✓
	CVE-2024-34359	CWE-76	llama-cpp-python	7				✓	✓	✓
	CVE-2025-1040	CWE-1336	AutoGPT	5				✓	✓	✓
	CVE-2025-24357	CWE-502	vLLM	15				✓	✓	✓
Request Forgery and Client Attacks	CVE-2024-1879	CWE-352	AutoGPT	5				✓	✓	✓
	CVE-2024-3095	CWE-918	LangChain	6				✓	✓	✓
	CVE-2024-10099	CWE-79	ComfyUI	3				✓	✓	✓
	CVE-2024-11822	CWE-918	dify	4				✓	✓	✓
	CVE-2025-64511	CWE-918	MaxKB	3				✓	✓	✓
	CVE-2025-65958	CWE-918	Open WebUI	8			✓	✓	✓	✓

Table 2: Overall detection performance on the benchmark.

Tools(Total: 30)	Detected	Missed	Detection-Rate
CodeQL	1	29	3.3%
Semgrep	0	30	0%
RepoAudit	6	24	20%
GPT-5.4+ManualCraft	19	11	63.3%
Claude Code	21	9	70%
SemScan	28	2	93.3%

318 lines for evidence constructor, 182 lines for the synthesis engine, and 356 lines for evidence set initialization and entry points.

Evaluation Scope. We target Python programs in our evaluation for the following reasons. First, Python is highly prevalent in modern open-source projects. 21 of the top 100 most-starred GitHub repositories are written in Python. Furthermore, it is challenging to analyze Python programs. Dynamic features such as runtime dispatch, indirect calls, and schema-driven (de)serialization through third-party libraries are common but rarely modeled by existing tools. Existing work on taint-style vulnerability detection for Python remains limited.

Benchmark Dataset. We construct a benchmark dataset of real-world CVE vulnerabilities from popular open-source Python projects. Specifically, we first collect Python-related CVEs disclosed between January 2023 and January 2026, and retain only projects with at least 1,000 GitHub stars. We further restrict the dataset to vulnerabilities with official security patches. These patches provide authoritative references for reconstructing and validating the ground-truth source-to-sink flows. From the resulting candidate pool, we randomly sample 30 CVEs for controlled evaluation. The details of these benchmark CVEs are listed in Table 1.

Real-World Dataset. To assess the practicality and scalability of SemScan to discover vulnerabilities in large-scale real-world

software, we construct a real-world project dataset comprising 200 popular Python repositories from GitHub. The projects are selected based on their star number at the time of collection, with forks and inactive repositories excluded to avoid duplicate programs and unmaintained projects.

Baseline Setup. We compare SemScan against five representative baselines, i.e., CodeQL [10] and Semgrep [30] as industrial static analyzers, RepoAudit [12] and Claude Code [1] as agent-based LLM approaches, and GPT-5.4+ManualCraft as an LLM baseline.

The agent-based baseline systems are selected based on two criteria. The first is support for Python, matching our target language. The second is applicability at the repository level, matching our real-world deployment setting. Among the LLM-based and agentic systems discussed in Section 5, IRIS and Magneto officially support only Java and do not meet the first criterion. LLMxCPG requires manual fragment-level decomposition of the target program, GRACE [24] utilizes CPG-to-text conversion that does not scale to full repository context, and Vul-RAG requires a pre-identified vulnerable snippet as input, precluding zero-day discovery. None of these three meet the second criterion. RepoAudit and Claude Code satisfy both criteria. Hence we include them as our baseline.

We then introduce the configuration of baseline tools. For RepoAudit, since its official implementation does not support several CWE categories in our benchmark, we manually construct source and sink extractors following the same vulnerability specification used by SemScan to ensure a fair comparison. To further control for model capability as a confounding factor, we configure RepoAudit to use the same backbone LLM as SemScan’s synthesis engine, i.e., DeepSeek-V3.2. For Claude Code, we equip it with the same repository access, tool infrastructure, and multi-round interaction setting as SemScan, but without evidence induction. For GPT-5.4+ManualCraft, we use manually crafted prompts. Due to

the context window limitation, it is difficult to directly send the full programs to the LLM. Instead, we manually identify vulnerability-related source files and provide them at the file level via prompts.

4.2 RQ1: Effectiveness in Benchmark

In this section, we evaluate the effectiveness of SemScan on the benchmark dataset and compare it with the five baseline tools specified in Section 4.1.

Result Overview. Table 2 summarizes the benchmark results. SemScan successfully detects 28 CVEs, achieving a detection rate of 93.3%. In comparison, CodeQL detects only 1 case (3.3%), Semgrep discovers 0 cases, RepoAudit finds 6 cases (20%), Claude Code uncovers 21 cases (70%), and GPT-5.4+ManualCraft detects 19 cases (63.3%). These results show that SemScan significantly outperforms all baselines on the benchmark.

SemScan Result Validation. We validate that SemScan’s 93.3% detection rate is statistically robust and generalizes to samples beyond the cutoff date, rather than being an artifact of memorization.

① *Reliability.* We quantify the statistical uncertainty of SemScan’s detection rate via a bootstrap analysis (200,000 resamples), yielding a 95% confidence interval of [83.3%, 100%] for the observed 93.3% detection rate. This interval remains substantially above all baseline detection rates reported in Table 2, confirming that SemScan’s advantage is statistically reliable rather than an artifact of the benchmark’s limited size.

② *Post-Cutoff Generalization Check.* To examine whether SemScan’s performance is driven by memorized CVE knowledge, we cross-check the benchmark against DeepSeek-V3.2’s training cutoff (May 2025). Among the 30 benchmark CVEs, six (CVE-2025-64511, CVE-2025-65958, CVE-2026-0761, CVE-2026-0766, CVE-2026-22807, and CVE-2026-24123) were disclosed after this cutoff date. SemScan detects all of them (100%). This indicates that SemScan’s effectiveness does not rely on pre-cutoff memorized vulnerability patterns.

SemScan Accuracy Analysis. Next, we check SemScan’s accuracy. In total, SemScan produces one false positive and has two false negatives (i.e., missing two annotated flows), corresponding to two CVE vulnerabilities.

CVE-2024-6331: This case contributes to both a false positive and a false negative. The vulnerability originates from user-controlled input received via a Socket.IO handler, which propagates through two branches depending on the runtime state. When `AgentState.is_agent_completed()` returns true, the flow reaches the true branch; otherwise, it reaches the false branch. Only the false directly triggers the vulnerability, while the true branch does not lead to an exploitable sink. This case exposes a known limitation of LLMs in multi-path planning. When several candidate paths are plausible, LLMs may prioritize one branch over others without systematically planning to explore all feasible alternatives. Consequently, SemScan reports the true branch path (counted as a false positive) while missing the actual vulnerable path through the false branch (counted as a false negative). We plan to address this in future work.

CVE-2024-10099: This case contributes to a false negative. The vulnerability arises from a two-stage interaction. An attacker first uploads a malicious file via `upload_image`, which is persisted on the server. The file is later served directly to the browser via the

`view_image` without Content-Type restriction, enabling client-side execution. Since the two stages are linked via the file system rather than explicit taint propagation, SemScan analyzes each stage in isolation and fails to compose them into a complete path, exposing a limitation in modeling cross-request, storage-mediated flows. Future Work will focus on strengthening SemScan to support cross-request taint paths.

Why Baselines Fail. At last, we investigate why the baseline tools fail in many cases. We find that these failures stem from the complex coding logic commonly present in real-world CVE programs. Specifically, as shown in Table 1, the CVE dataset involves call depths ranging from 2 to 15, with an average of 6.1, indicating significant interprocedural complexity. Moreover, the dynamic Python features are prevalent in real-world programs, including higher-order functions, closures, and dynamically-determined types. As a result, the static analyzers CodeQL and Semgrep struggle to handle these real-world vulnerabilities, leading to high false negatives of missed vulnerabilities. For RepoAudit, as discussed in section 1, it tends to lose track in fragmented semantic evidence and fails to capture global semantics, making it difficult to handle long chains (e.g., chain lengths exceeding 4 function calls). In contrast, GPT-5.4+ManualCraft performs much better, highlighting the effectiveness of LLMs. However, it relies heavily on manually crafted prompts, which require significant manual effort to identify and extract minimized vulnerability-relevant code snippets from the target large-scale program. These prompts assume that the vulnerabilities are already known and that relevant files have been identified in advance, limiting scalability and practicality. For Claude Code, it achieves best performance among baselines with 21 CVEs detected. But without evidence induction and re-synthesis, it fails to accumulate security-relevant evidence accurately and sustains correctness over long inter-procedural chains. Therefore, evidence set is insufficient and early errors are propagated to subsequent reasoning, leading to incomplete paths that lack important taint propagation or completely different taint paths that can not be exploited.

4.3 RQ2: Model Robustness and Scaling

In this section, we analyze the robustness and scaling behavior of SemScan on the benchmark by decoupling the framework into two components: the synthesis engine and the evidence constructor. We evaluate detection performance using two metrics: constructed data-flow steps and vulnerability detection. ‘Detected’ indicates whether a source-to-sink flow is successfully identified for a benchmark case, and it captures whether a vulnerability-relevant flow is discovered. ‘Step-coverage’ measures the correctly recovered data-flow steps over the total number of annotated steps in the ground-truth path, and it further quantifies the completeness of the recovered flow.

Table 3 summarizes the detection performance across all model configurations. First, SemScan becomes stable once the evidence constructor reaches a moderate scale. Scaling the constructor from Qwen2.5-3B-inst to Qwen2.5-7B-inst improves vulnerability detection from 19 to 28 and step-coverage from 222 to 314. In contrast, further scaling to 14B does not provide a stable gain, with overall performance dropping to 26 detected vulnerabilities and 266 step-coverage. Second, the choice of synthesis engine has a smaller effect.

Table 3: Sensitivity Analysis of Evidence Constructor Scaling and Synthesis Engine Robustness

CVE ID	Evidence Constructor Models						Synthesis Engine Models					
	Qwen2.5-3B-inst		Qwen2.5-7B-inst		Qwen2.5-14B-inst		DeepSeek-V3.2		GPT-5.4		Qwen3.6-plus	
	Step-Coverage	Detected	Step-Coverage	Detected	Step-Coverage	Detected	Step-Coverage	Detected	Step-Coverage	Detected	Step-Coverage	Detected
CVE-2024-1881	12/16	✓	13/16	✓	10/16	✓	13/16	✓	11/16	✓	16/16	✓
CVE-2024-3271	N/A	N/A	13/13	✓	12/13	✓	13/13	✓	13/13	✓	13/13	✓
CVE-2025-25362	7/7	✓	7/7	✓	7/7	✓	7/7	✓	7/7	✓	7/7	✓
CVE-2026-0761	N/A	N/A	8/8	✓	8/8	✓	8/8	✓	8/8	✓	8/8	✓
CVE-2026-0766	4/4	✓	4/4	✓	4/4	✓	4/4	✓	4/4	✓	4/4	✓
CVE-2026-22807	10/13	✓	11/13	✓	8/13	✓	11/13	✓	10/13	✓	11/13	✓
CVE-2024-6331	5/18	✗	5/18	✗	5/18	✗	5/18	✗	5/18	✗	5/18	✗
CVE-2024-11958	8/9	✓	8/9	✓	8/9	✓	8/9	✓	8/9	✓	7/9	✓
CVE-2024-12909	N/A	N/A	8/8	✓	8/8	✓	8/8	✓	8/8	✓	8/8	✓
CVE-2024-23731	21/25	✓	22/25	✓	22/25	✓	22/25	✓	20/25	✓	19/25	✓
CVE-2024-29374	7/7	✓	7/7	✓	7/7	✓	7/7	✓	7/7	✓	7/7	✓
CVE-2023-51449	3/5	✓	4/5	✓	3/5	✓	4/5	✓	4/5	✓	3/5	✓
CVE-2024-4941	N/A	N/A	9/22	✓	7/22	✓	9/22	✓	10/22	✓	9/22	✓
CVE-2024-8953	14/14	✓	14/14	✓	9/14	✓	14/14	✓	11/14	✓	10/14	✓
CVE-2024-8958	9/11	✓	10/11	✓	7/11	✗	10/11	✓	9/11	✓	10/11	✓
CVE-2024-11042	N/A	N/A	11/13	✓	11/13	✓	11/13	✓	8/13	✓	8/13	✓
CVE-2024-28088	N/A	N/A	6/11	✓	6/11	✓	6/11	✓	11/11	✓	6/11	✓
CVE-2026-24123	17/19	✓	19/19	✓	14/19	✓	19/19	✓	15/19	✓	17/19	✓
CVE-2023-6730	7/9	✓	9/9	✓	7/9	✓	9/9	✓	7/9	✓	8/9	✓
CVE-2023-7018	10/15	✓	12/15	✓	9/15	✓	12/15	✓	9/15	✓	10/15	✓
CVE-2024-9053	N/A	N/A	8/8	✓	7/8	✓	8/8	✓	8/8	✓	6/8	✓
CVE-2024-34359	11/18	✗	13/18	✓	8/18	✗	13/18	✓	12/18	✓	12/18	✓
CVE-2025-1040	9/9	✓	9/9	✓	9/9	✓	9/9	✓	9/9	✓	9/9	✓
CVE-2025-24357	15/24	✓	13/24	✓	8/24	✓	13/24	✓	7/24	✓	7/24	✓
CVE-2024-1879	11/11	✓	11/11	✓	10/11	✓	11/11	✓	11/11	✓	11/11	✓
CVE-2024-3095	N/A	N/A	10/10	✓	10/10	✓	10/10	✓	10/10	✓	10/10	✓
CVE-2024-10099	10/10	✗	10/10	✗	10/10	✗	10/10	✗	9/10	✗	9/10	✗
CVE-2024-11822	13/17	✓	17/17	✓	14/17	✓	17/17	✓	15/17	✓	14/17	✓
CVE-2025-64511	9/11	✓	9/11	✓	9/11	✓	9/11	✓	11/11	✓	10/11	✓
CVE-2025-65958	10/29	✓	14/29	✓	9/29	✓	14/29	✓	29/29	✓	6/29	✗
Total	222/394	19/30	314/394	28/30	266/394	26/30	314/394	28/30	306/394	28/30	280/394	27/30

N/A indicates failure due to context window limits or timeout.

DeepSeek-V3.2 and GPT-5.4 both detect 28 cases, with similar step-coverage values of 314 and 306, respectively, while Qwen3.6-plus remains competitive at 27 detected vulnerabilities and 280 steps covered. These results are expected given SemScan’s decomposed architecture (Section 3.3). The evidence constructor only needs to reach a capability threshold for reliable local extraction, while the synthesis engine is largely interchangeable across sufficiently capable models. The 7B model already meets this threshold, achieving near-optimal performance, whereas scaling to 14B yields no consistent gains.

Synthesis Engine Robustness. We evaluate three state-of-the-art models as the synthesis engine: DeepSeek-V3.2, GPT-5.4, and Qwen3.6-plus. As shown in Table 3, all three models achieve comparable detection performance and step-coverage across most benchmark cases. GPT-5.4 performs slightly better on a few complex cases (e.g., CVE-2025-65958), but the overall differences are limited. We attribute this robustness to SemScan’s evidence-grounded design. Once reliable local evidence is constructed, the synthesis stage mainly performs structured aggregation rather than complex code reasoning, reducing sensitivity to model choice. Therefore, any sufficiently capable model can perform consistently given proper instruction following and global reasoning over the accumulated evidence.

Evidence Constructor Scaling Behavior. We investigate the impact of evidence constructor size using the Qwen2.5 family at three scales: 3B, 7B, and 14B. As shown in Table 3, the 3B model frequently fails to produce valid outputs, often due to instruction-following errors that exhaust the context window (e.g., CVE-2024-4941). Increasing model size to 7B eliminates most of these failures and substantially improves the number of detected cases, reaching 28.

Further scaling to 14B does not yield consistent gains, and in some cases even reduces step-coverage, possibly because larger models tend to favor higher-level abstraction over explicit step-by-step evidence extraction. These results indicate a capability threshold for the evidence constructor. Once the model is strong enough for reliable local evidence extraction, further scaling provides limited benefit.

Based on the above analysis, we select DeepSeek-V3.2 as the synthesis engine and Qwen2.5-7B-Instruct as the evidence constructor for all experiments. DeepSeek-V3.2 achieves the best overall vulnerability detection performance and step-coverage among the evaluated synthesis engines while remaining cost-competitive. Qwen2.5-7B-Instruct meets the capability threshold for reliable local evidence extraction, beyond which further scaling yields no consistent gains.

4.4 RQ3: Ablation Study

In this section, we validate the architectural design of SemScan on the benchmark by comparing it against two ablated variants: (1) *w/o gap-guided*, which removes the gap-guided evidence expansion mechanism and relies only on the initial context and unguided retrieval; and (2) *w/o Re-synthesis*, which disables iterative re-synthesis and instead continuously extends partial paths produced in previous rounds. We use case-level recall as the evaluation metric in this ablation study.

To better understand how the effectiveness of each component varies with vulnerability complexity, we partition the 30 benchmark cases into three levels based on *Depth*: Low ($\text{Depth} \leq 4$), Medium ($\text{Depth} 5\text{--}8$), and High ($\text{Depth} \geq 9$).

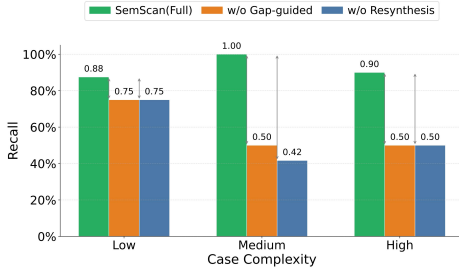


Figure 3: Impact of architectural components on path recovery rate across complexity levels.

As shown in Figure 3, removing either component degrades recall, and the degradation becomes more pronounced as vulnerability complexity increases. For low complexity cases, recall drops from 88% to 75% for both variants. For medium complexity cases, recall drops from 100% to 50% without gap-guided evidence expansion and further to 42% without re-synthesis. For high complexity cases, recall drops from 90% to 50% in both variants. These results show that both gap-guided evidence expansion and re-synthesis become increasingly important for medium and high complexity vulnerabilities, where long-chain dependencies and incomplete evidence sets are more common.

Effectiveness of Gap-Guided Expansion. Removing the gap-guided evidence expansion mechanism leads to a clear drop in recall, especially for medium and high complexity vulnerabilities. Specifically, recall decreases from 100% to 50% on medium complexity cases and from 90% to 50% on high complexity cases. By comparison, the drop in low complexity cases is smaller, from 88% to 75%. These results show that unguided retrieval based only on static local context is often insufficient for recovering complete vulnerability paths once the required evidence becomes fragmented across files or reasoning steps. Gap-guided evidence expansion is effective because it steers evidence collection toward program regions that are specifically needed to close unresolved gaps, rather than retrieving additional context without guidance.

Effectiveness of Iterative Re-synthesis. Disabling re-synthesis also causes substantial degradation, particularly for medium and high complexity vulnerabilities. Recall drops from 100% to 42% on medium complexity cases and from 90% to 50% on high complexity cases, while the drop on low complexity cases is more limited, from 88% to 75%. This pattern indicates that the benefit of re-synthesis grows with reasoning depth. For low complexity vulnerabilities, where the data-flow path is short and weakly interdependent, directly extending previously constructed partial paths is often sufficient. For more complex cases, however, partial paths built under incomplete evidence can contain early mistakes or omissions, and these errors are then propagated and amplified in later rounds if the system only continues extension. Re-synthesis mitigates this problem by reconstructing candidate paths from scratch over the accumulated evidence at each iteration, thereby improving the stability and global consistency of long-chain vulnerability detection.

Table 4: Baseline coverage of the 62 vulnerabilities detected by SemScan on the real-world dataset.

Tool	Detected	Missed	Coverage Rate
CodeQL	8	54	12.90%
Semgrep	1	61	1.61%
RepoAudit	13	49	20.97%
GPT-5.4+ManualCraft	34	28	54.84%

4.5 RQ4: Real-World Vulnerability Discovery

We conducted the evaluation on the wild dataset described in Section 4.1. For each project, we employ CodeQL to automatically extract candidate sources and sinks, which provide broad coverage of common vulnerability patterns in Python programs. To improve specification quality, we manually filter out low-quality entries, such as those whose symbols are "f-string" or "Attr()".

SemScan reports 106 potential vulnerabilities on the real-world dataset. 62 of them have been manually confirmed as true positives through PoC construction and verification, yielding a precision of 58.49% over all reported cases. These confirmed findings affect 39 projects spanning diverse application domains, including web frameworks, data processing pipelines, and AI infrastructure components. All affected projects have at least 1,000 GitHub stars, with 26 exceeding 10,000 stars. The confirmed findings are dominated by high-impact vulnerability classes such as code execution, injection, and deserialization-related flaws. All 62 confirmed findings have been responsibly disclosed to the corresponding project maintainers, of which 22 have been assigned CVE identifiers.

Confirmation Protocol. All vulnerabilities were confirmed by PoCs. Four experienced security experts (also co-authors) were involved in PoC generation and confirmation. Confirmers were only given SemScan’s reported source-sink taint path (necessary to construct the PoC) without SemScan’s reasoning trace to avoid judgment bias. Only when a security flaw was dynamically triggered by a PoC was it treated as a confirmed vulnerability. Ambiguous cases were independently re-verified by a second expert.

False Positives. We manually inspected the reports and found two main reasons for false positives. Most FPs (31/44, 70.45%) stem from the over-approximation of the CodeQL source/sink specifications, which are the starting points of our SemScan approach. The CodeQL-derived source and sink specifications often over-approximate realistic exploit conditions even after manual filtering. Some sources are not externally controllable, and some sinks lack required contextual conditions, such as shell-interpreted execution in subprocess.Popen(). The remaining FPs (13/44, 29.55%) stem from SemScan’s limited sensitivity to protective logic such as validation, allow-list checks, and sanitization, especially when it is off the main source-to-sink path or hidden behind helper functions and framework abstractions. Consequently, SemScan may conservatively report reachable paths that are not exploitable.

Comparison with Baselines. To examine whether existing tools can detect the 62 confirmed vulnerabilities discovered by SemScan, we compare CodeQL, Semgrep, RepoAudit, and GPT-5.4+ManualCraft under the setup described in Section 4.1. A finding is counted as covered only when the baseline reports a source-to-sink propagation that reaches a semantically equivalent security-sensitive sink.

As shown in Table 4, CodeQL, Semgrep, and RepoAudit cover only 8, 1, and 13 of the 62 confirmed vulnerabilities, respectively. GPT-5.4+ManualCraft achieves the highest baseline coverage, detecting 34 cases, but still misses 28 confirmed vulnerabilities. Many of these vulnerabilities involve cross-file dependencies, implicit data transformations, or dynamically resolved behaviors that require reasoning beyond the local scope of individual functions or files. This explains their consistently low coverage and highlights the advantage of SemScan’s ability to extend LLMs’ local reasoning abilities to global-view taint analysis.

4.6 RQ5: Performance and Cost Efficiency

We evaluate SemScan’s operational feasibility in terms of latency, token consumption, and monetary cost, reporting per-CVE averages on the benchmark and per-repository averages on the real-world dataset.

Results. As shown in Table 5, SemScan completes each benchmark case in 6.2 minutes on average, consuming 234,697 tokens and costing \$0.065 per case, corresponding to \$0.070 per detected vulnerability. On the real-world project dataset, SemScan analyzes each repository in 9.0 minutes on average, with an average cost of \$0.096 per repository and \$0.309 per confirmed vulnerability.

The low overhead mainly comes from the decomposed architecture of SemScan. Lightweight evidence constructors are executed in parallel for localized evidence extraction, while the more capable synthesis engine is reserved for global reasoning over accumulated evidence. This design reduces unnecessary large-model invocations and keeps monetary cost manageable when scaling to hundreds of real-world projects. Compared with manual security auditing of projects at this scale, the amortized cost remains low.

Table 5: Performance and cost overview of SemScan across benchmark and real-world settings.

Metric	Benchmark	Real-World
Projects / Cases Analyzed	30	200
Avg. Latency per Case (min)	6.2	9.0
Avg. Synthesis Engine Tokens	202,763	302,756
Avg. Evidence Constructor Tokens	31,934	85,326
Avg. Cost per Case (USD)	\$0.065	\$0.096
Vulnerabilities Found	28/30	62
Avg. Cost per Vuln. (USD)	\$0.070	\$0.309

4.7 Case Study

4.7.1 Vulnerability Discovery in AI Generation Infrastructure. This case is drawn from an open-source AI infrastructure project with over 100,000 stars on GitHub. SemScan successfully inspected its asynchronous request handling pipeline and uncovered an insecure deserialization vulnerability that traditional static analysis failed to trace. The relevant execution flow is illustrated in Figure 4.

The vulnerability manifests in a multi-stage, decoupled data-processing pipeline. The function `post_prompt()` (Line 1) serves as the entry point, receiving a user-controlled request. The extracted prompt is placed into an internal task queue, which is subsequently consumed by a worker process in a separate execution context. The flow then reaches the `execute()` (Line 9) method, where the tool must resolve a highly dynamic dispatch: the code utilizes a mapping

```

1 async def post_prompt(request):
2     prompt = (await request.json())["prompt"]
3     self.prompt_queue.put(prompt)

4 def prompt_worker(q, server_instance):
5     e = execution.PromptExecutor()
6     queue_item = q.get(timeout=timeout)
7     if queue_item is not None:
8         e.execute(queue_item)

9 async def execute(dynprompt):
10    class_type = dynprompt["class_type"]
11    class_def = CLASS_MAPPINGS[class_type]
12    obj = class_def()
13    f = getattr(obj, "load_checkpoint")
14    f(dynprompt["params"])

15 class CheckpointLoaderSimple:
16     def load_checkpoint(self, ckpt_name):
17         ckpt_path = torch.load(ckpt_name)

```

Figure 4: Case 1 of Insecure Deserialization

```

1 @routes.post("/api/jobs/")
2 async def submit_job(self, req):
3     resp = await submit_job_internal(req)

4 async def submit_job_internal(self, req):
5     async with self.session.post(
6         f"{self.address}/api/job_agent/jobs/",
7         json=dataclasses.asdict(req)
8     ) as resp:
9         if resp.status == 200:
10            result_json = await resp.json()
11            return JobSubmitResponse(**result_json)

12 @routes.post("/api/job_agent/jobs/")
13 async def submit_job(self, req):
14     _id = self.get_job_manager().submit_job(
15         entrypoint=req.entrypoint, ... )

16 class JobManager:
17     async def submit_job(self, entrypoint, ...):
18         supervisor = Supervisor(entrypoint, ...)
19         supervisor.run.remote()

20 class Supervisor:
21     async def run(self):
22         child_process = subprocess.Popen(
23             self.entrypoint, shell=True, ...)

```

Figure 5: Case 2 of Command Injection

dictionary to instantiate a class based on the attacker-supplied `class_type`, and subsequently invokes the `load_checkpoint` (Line 17) method via reflexive attribute access. This eventually leads to the sensitive sink `torch.load()`, where the file path is derived directly from the external input. Since `torch.load` utilizes `pickle` for deserialization by default, an attacker can provide a malicious payload to execute arbitrary commands upon loading, leading to full system compromise.

4.7.2 Cross-Component Semantic Reasoning in Distributed Systems. This case study highlights a critical vulnerability identified in a popular open-source distributed computing framework with over 10,000 stars on GitHub. SemScan uncovered an end-to-end command injection path that spans multiple network boundaries and service components, as shown in Figure 5. The data flow originates from a job submission entry point. Our tool tracked the untrusted `entrypoint` parameter as it was passed into internal submission logic and subsequently forwarded via an asynchronous HTTP request to a remote agent.

The primary analysis challenge involves the cross-component propagation and the subsequent remote execution. Upon receiving the forwarded request, the system dispatches the task to a supervisor instance, which invokes a remote execution primitive. SemScan leveraged its LLM-based semantic reasoning to recognize

that the entrypoint attribute (originally sourced from the external API request) was ultimately fed into a subprocess.Popen call with shell=True. By bridging the semantic gap between the web-facing API and the remote execution worker, SemScan successfully identified an RCE flaw. Traditional static analysis tools typically lose context during internal HTTP forwarding or at the propagation to remote actor execution, whereas SemScan maintains taint persistence by contextually linking these distributed components.

5 Related Work

Traditional Vulnerability Detection. Traditional vulnerability detection techniques can be categorized into static analysis and dynamic analysis. Among static analysis tools, Semgrep [30] detects vulnerabilities by matching predefined rules against source code. Jern [8] represents programs as code property graphs (CPG) and supports vulnerability discovery through graph queries. CodeQL [10] models programs as queryable databases and enables declarative security analyses, including taint tracking. However, the effectiveness of static approaches is bounded by predefined rules or underlying program models, making it difficult to capture vulnerabilities that depend on implicit or runtime-mediated semantics. Among dynamic analysis techniques, AFL [39] uses coverage-guided fuzzing to expose crashes through randomized execution. KLEE [3] applies symbolic execution to systematically explore program paths and generate high-coverage test cases. Driller [34] combines fuzzing with selective symbolic execution to overcome path exploration barriers. While effective in discovering execution-triggered vulnerabilities, dynamic approaches remain constrained by limited path coverage and the complexity of environment setup [16].

Deep Learning-Based Security Vetting. Existing deep learning-based vulnerability detection methods can be broadly categorized into two paradigms: sequence-based and graph-based approaches. Among sequence-based methods, VulDeepecker [21] firstly represents programs as code gadgets based on slicing and uses BiLSTM for vulnerability detection. Following this direction, GNNs (Graph Neural Network) are introduced. Devign [42] represents code as a composite graph (AST, CFG, data flow) and applies gated graph neural networks for classification. However, these approaches rely on localized, pre-constructed program representations, limiting their ability to recover long-chain, cross-function dependencies that require global information acquisition.

LLM-Assisted Security Analysis. Research in this area focuses on applying LLMs to assist program analysis tools. In static analysis, IRIS [20] employs LLMs to assist CodeQL to reduce false negatives and false positives in taint-style Java vulnerability detection. Lift [18] utilizes LLMs to reason about complex branch conditions and path constraints, addressing challenging UBI cases. In dynamic analysis, Magneto [43] decomposes call chains and leverages LLMs to generate test inputs. However, these approaches rely on external program analysis to supply intermediate representations, which constrains LLM reasoning to local context and limits their ability to recover incomplete taint paths when the underlying analysis fails to capture critical program behaviors.

LLM-Centric Vulnerability Hunting. Research in this area can be further divided into two directions. One direction focuses on

enhancing LLMs through task-specific post-training. VulLLM [6] decomposes vulnerability detection into three subtasks and improves performance through multi-instruction fine-tuning. VulnLLM-R [28] adapts smaller open-source models into specialized vulnerability detectors through supervised fine-tuning and preference optimization. However, such approaches rely heavily on large-scale high-quality labeled datasets and incur substantial training costs.

Another direction leverages the reasoning capabilities of LLMs directly. Prompting-based works [4, 9, 23] embed code snippets into prompts directly and require models to identify vulnerabilities. GRACE [24] converts CPGs into natural language representations to guide LLM reasoning. LLMxCPG [17] performs CPG-based slicing to extract vulnerability-relevant code regions before applying LLM-based analysis. These approaches operate on fixed code contexts or pre-extracted program representations, limiting their ability to adaptively explore programs and recover missing semantic connections required for complete vulnerability reasoning.

Agent-based systems further extend LLMs' capabilities by combining LLMs with techniques such as RAG and pre-defined tools. Vul-RAG [7] builds a vulnerability knowledge base to provide relevant vulnerability information to LLMs. RepoAudit [12] employs an LLM-based agent to explore interprocedural data flows starting from source values via value propagation. Their exploration remains local and lacks mechanisms for global planning or recovering missing semantic information, limiting the reconstruction of real-world vulnerability flows.

6 Discussion

LLM Pre-training Data Contamination. A potential concern is whether SemScan benefits from memorized vulnerability patterns. This risk is mitigated because SemScan receives only repository code and semantic evidence without CVE identifiers, descriptions, patches, or disclosure metadata, and correctness requires a structurally valid source-to-sink path grounded in the analyzed code. Moreover, performance on CVEs disclosed after the synthesis engine's training cutoff remains consistent with that on the full benchmark (Section 4.2). The undisclosed zero-day vulnerabilities confirmed in our wild evaluation provide further evidence independent of publicly documented vulnerability reports (Section 4.5).

Limitations and Future Work. Despite its effectiveness on large-scale Python projects, SemScan has several limitations. First, it relies on externally provided source and sink specifications, currently extracted using static analysis tools such as CodeQL, whose incompleteness and over-approximation can affect detection. On our CVE benchmark, CodeQL recovers only 17/30 source-sink pairs, while its over-approximation accounts for most false positives in the wild evaluation (Section 4.5). Second, SemScan may miss vulnerable paths when multiple feasible branches require exhaustive exploration. Third, it cannot currently compose taint flows spanning multiple HTTP requests through persistent storage. Finally, SemScan focuses on taint-style vulnerabilities in Python. Future work will address these limitations through more precise source/sink extraction, improved branch exploration, cross-request taint reasoning, and support for additional languages and vulnerability classes.

7 Conclusion

In this paper, we propose SemScan, a novel LLM-based framework for detecting taint-style vulnerabilities in real-world large-scale Python programs. By introducing the evidence induction theory with iterative synthesis, SemScan successfully extends local analysis capabilities of LLMs to global-view security analysis. This design overcomes the fundamental limitations faced by both traditional static analyzers and prior LLM-based approaches. Our experiments on 200 popular open-source real-world Python projects show that SemScan effectively uncovers 62 confirmed 0-day vulnerabilities across 39 real-world projects, of which 22 have been assigned CVE identifiers. We believe SemScan and its evidence induction idea can serve as a foundation for future work on automated, scalable security auditing of real-world software.

Acknowledgments

We would like to thank the anonymous reviewers and those who provided valuable comments and suggestions that helped improve the quality of this paper. This work was supported in part by the National Natural Science Foundation of China under Grant Nos. U25A6023 and 62202106.

References

- [1] Anthropic. 2025. Claude Code: AI coding assistant. <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>. Accessed: 2026-07-01.
- [2] Enna Basic and Alberto Giarretta. 2025. From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security. Accessed: 2026-04-27. arXiv:2412.15004
- [3] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [4] Anton Cheshkov, Pavel Zadorozhny, and Rodion Levichev. 2023. Evaluation of ChatGPT Model for Vulnerability Detection. Accessed: 2026-04-27. arXiv:2304.07232
- [5] Xueying Du, Jiayi Feng, Yi Zou, Wei Xu, Jie Ma, Wei Zhang, Sisi Liu, Xin Peng, and Yiling Lou. 2026. Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry. Accessed: 2026-04-27. arXiv:2601.18844
- [6] Xiaohu Du, Ming Wen, Jiahao Zhu, Zifan Xie, Bin Ji, Huijun Liu, Xuanhua Shi, and Hai Jin. 2024. Generalization-Enhanced Code Vulnerability Detection via Multi-Task Instruction Fine-Tuning. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [7] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2026. VulRAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2026). doi:10.1145/3797277
- [8] Fraunhofer AISEC. 2026. Joern: Code Analysis Platform. <https://joern.io/>. Accessed: 2026-04-27.
- [9] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?. In *Proceedings of the Asia-Pacific Software Engineering Conference (APSEC)*. doi:10.1109/APSEC60848.2023.00085
- [10] GitHub Inc. 2026. CodeQL. <https://github.com/github/codeql>. Accessed: 2026-04-27.
- [11] Ethan Goh, Robert J. Gallo, Eric Strong, Yingjie Weng, Hannah Kerman, Jason A. Freed, Joséphine A. Cool, Zahir Kanjee, Kathleen P. Lane, Andrew S. Parsons, Neera Ahuja, Eric Horvitz, Daniel Yang, Arnold Milstein, Andrew P. J. Olson, Jason Hom, Jonathan H. Chen, and Adam Rodman. 2025. Publisher Correction: GPT-4 assistance for improvement of physician performance on patient care tasks: a randomized controlled trial. *Nature Medicine* (2025). doi:10.1038/s41591-025-03586-x
- [12] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. 2025. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing. In *International Conference on Machine Learning (ICML)*.
- [13] Yuejun Guo, Constantinos Patsakis, Qiang Hu, Qiang Tang, and Fran Casino. 2024. Outside the Comfort Zone: Analysing LLM Capabilities in Software Vulnerability Detection. In *European Symposium on Research in Computer Security (ESORICS)*.
- [14] Yiwei Hu, Zhen Liu, Kedie Shu, Shenghua Guan, Deqing Zou, Shouhuai Xu, Bin Yuan, and Hai Jin. 2025. SoK: Automated Vulnerability Repair: Methods, Tools, and Assessments. In *USENIX Security Symposium*.
- [15] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2026).
- [16] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. doi:10.1145/3243734.3243804
- [17] Ahmed Lekssays, Hamza Mouhcine, Khang Tran, Ting Yu, and Issa Khalil. 2025. LLMxCPG: context-aware vulnerability detection through code property graph-guided large language models. In *USENIX Security Symposium* (Seattle, WA, USA). Article 26, 19 pages.
- [18] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. In *ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. doi:10.1145/3649828
- [19] Yue Li, Xiao Li, Hao Wu, Minghui Xu, Yue Zhang, Xiuzhen Cheng, Fengyuan Xu, and Sheng Zhong. 2025. Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask. Accessed: 2026-04-27. arXiv:2504.13474
- [20] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=9LdJDU7E91>
- [21] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2018.23158
- [22] Hanmeng Liu, Zhizhang Fu, Mengru Ding, Ruoxi Ning, Chaoli Zhang, Xiaozhang Liu, and Yue Zhang. 2025. Logical Reasoning in Large Language Models: A Survey. Accessed: 2026-04-27. arXiv:2502.09100
- [23] Xin Liu, Yuan Tan, Zhenghang Xiao, Jianwei Zhuge, and Rui Zhou. 2023. Not The End of Story: An Evaluation of ChatGPT-Driven Vulnerability Description Mappings. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. <https://aclanthology.org/2023.findings-acl.229/>
- [24] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software* (2024). doi:10.1016/j.jss.2024.112031
- [25] Ziming Luo, Zonglin Yang, Zexin Xu, Wei Yang, and Xinya Du. 2025. LLM4SR: A Survey on Large Language Models for Scientific Research. Accessed: 2026-04-27. arXiv:2501.04306
- [26] Yisroel Mirsky, George Macon, Michael Brown, Carter Yagemann, Matthew Pruett, Evan Downing, Sukarno Mertoguno, and Wenke Lee. 2023. VulChecker: graph-based vulnerability localization in source code. In *USENIX Security Symposium*. Article 367, 18 pages.
- [27] MITRE Corporation. 2026. CVE-2026-24123. <https://nvd.nist.gov/vuln/detail/CVE-2026-24123>. Accessed: 2026-04-27.
- [28] Yuzhou Nie, Hongwei Li, Chengquan Guo, Ruizhe Jiang, Zhun Wang, Bo Li, Dawn Song, and Wenbo Guo. 2025. VulLLM-R: Specialized Reasoning LLM with Agent Scaffold for Vulnerability Detection. Accessed: 2026-04-27. arXiv:2512.07533
- [29] Libo Qin, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang, Yinghui Li, Min Li, Wanxiang Che, and Philip S. Yu. 2026. Large language models meet NLP: a survey. *Frontiers of Computer Science* (2026), 23 pages. doi:10.1007/s11704-025-50472-3
- [30] ReturnToCorp. 2024. Semgrep: Lightweight Static Analysis for Security and Correctness. <https://semgrep.dev>. Accessed: 2026-04-27.
- [31] Kumar Shashwat, Francis Hahn, Xinming Ou, Dmitry Goldgof, Lawrence Hall, Jay Ligatti, S. Raj Rajagopalan, and Armin Ziaie Tabari. 2024. A Preliminary Study on Using Large Language Models in Software Penesting. In *Proceedings of the Workshop on SOC Operations and Construction (WOSOC)*. <https://www.ndss-symposium.org/ndss-paper/auto-draft-524/>
- [32] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *Comput. Surveys* (2025). doi:10.1145/3769082
- [33] Benjamin Steenhoeck, Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam, Hengbo Tong, Swarna Das, Earl T. Barr, and Wei Le. 2025. To Err is Machine: Vulnerability Detection Challenges LLM Reasoning. Accessed: 2026-04-27. arXiv:2403.17218
- [34] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In *Network and Distributed System Security Symposium (NDSS)*.
- [35] Yaou Wu, Dongyuan Li, Yankai Chen, Renhe Jiang, Henry Peng Zou, Wei-Chieh Huang, Yangning Li, Liancheng Fang, Zhen Wang, and Philip S. Yu. 2025. Multi-Agent Autonomous Driving Systems with Large Language Models: A Survey of Recent Advances, Resources, and Future Directions. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. doi:10.18653/v1/2025.findings-

- emnlp.683
- [36] Shangzhi Xu, Jialiang Dong, Weiting Cai, Juanru Li, Arash Shaghghi, Nan Sun, and Siqi Ma. 2025. Enhancing Security in Third-Party Library Reuse - Comprehensive Detection of 1-day Vulnerability through Code Patch Analysis. In *Network and Distributed System Security Symposium (NDSS)*.
- [37] Yaopeng Yang, Chuanyi Li, Zhifeng Han, Rui Li, Kui Xu, Qingyuan Li, Wenkang Zhong, Zongwen Shen, Zhiwei Fei, Jidong Ge, and Bin Luo. 2025. Patch Generation in APR: A Survey from the Perspectives of Utilizing LLMs and Using APR-Specific Information. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2025). <https://doi.org/10.1145/3764584>
- [38] Alperen Yildiz, Sin G Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil Mon Divakaran. 2025. Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. doi:10.18653/v1/2025.acl-long.1490
- [39] Michał Zalewski. 2026. American Fuzzy Lop (AFL) Technical Whitepaper. <http://lcamtuf.coredump.cx/afl/>. Accessed: 2026-04-27.
- [40] Da Zheng, Lun Du, Junwei Su, Yuchen Tian, Yuqi Zhu, Jintian Zhang, Lanning Wei, Ningyu Zhang, and Huajun Chen. 2025. Knowledge Augmented Complex Problem Solving with Large Language Models: A Survey. Accessed: 2026-04-27. arXiv:2505.03418
- [41] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, Article 145 (2025), 31 pages. <https://doi.org/10.1145/3708522>
- [42] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [43] Zhuotong Zhou, Yongzhuo Yang, Susheng Wu, Yiheng Huang, Bihuan Chen, and Xin Peng. 2024. Magneto: A Step-Wise Approach to Exploit Vulnerabilities in Dependent Libraries via LLM-Empowered Directed Fuzzing. In *International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3691620.3695531
- [44] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large Language Models for Information Retrieval: A Survey. *ACM Transactions on Information Systems* (2025). doi:10.1145/3748304

A Open Science

We have open-sourced the implementation of SemScan at the GitHub repository <https://github.com/FDU-SSR/SemScan> to support further research and facilitate independent evaluation. We hope that the released source code will encourage broader experimentation and provide a useful foundation for future work on real-world vulnerability detection.

B Ethical Considerations

This work is limited to defensive vulnerability detection. All experiments used publicly available CVEs and open-source software in offline, controlled environments, without accessing or interfering with live systems, private repositories, proprietary code, or personal data. Newly identified vulnerabilities are handled through responsible disclosure before public release. When third-party LLM APIs were used, only code necessary for semantic reasoning was submitted under the providers’ privacy policies, with non-essential identifying metadata removed locally. The system does not support exploit generation or unauthorized access. Accordingly, we identify no material ethical or security concerns beyond those inherent to standard defensive vulnerability research.

C Newly Discovered Vulnerabilities

Table 6 summarizes the 62 new vulnerabilities discovered by SemScan in popular Open Source Python projects, detailing their type and potential exploit difficulty.

Table 6: Confirmed 0-day vulnerabilities in 200 Python projects.

Project	Stars	LoC	Vulnerability-Count	Vulnerability-Type
langflow	147k	137,684	1	Code Execution
mem0	53k	2,896,123	3	Incorrect Access Control Incorrect Access Control Incorrect Access Control
ray	42k	1,069,478	1	Code Execution
qlib	41k	42,922	1	Deserialization
mindsdb	39k	134,332	1	Injection
gpt-pilot	33k	15,506	1	OS Command Injection
dspy	33k	1,261,558	2	Deserialization
pytorch-lightning	31k	87,654	2	Deserialization
flash-attention	23k	62,709	1	Deserialization
IOPaint	23k	3,745,821	1	Deserialization
CosyVoice	20k	12,348	3	Deserialization
magenta	19k	976,521	1	Deserialization
SWE-agent	19k	1,726,359	2	Injection
kirara-ai	18k	20,436	2	Access Control Code Execution
SuperAGI	17k	21,600	1	Deserialization
ml-engineering	17k	2,685	1	Deserialization
codellama	16k	929	1	Deserialization
cognee	16k	1,956,965	1	Code Execution
dgl	14k	2,781,565	1	Injection
flair	14k	3,250,002	2	Deserialization
horovod	14k	43,890	1	Deserialization
imgaug	14k	93,437	1	Deserialization
TinyZero	13k	20,392	1	Code Injection
PaddleSpeech	12k	115,538	1	Code Execution
OpenLLM	12k	1,747	1	Code Execution
ludwig	11k	95,503	2	Deserialization Deserialization
llm	11k	13,747	1	Code Injection
petals	10k	7,859	1	Deserialization
PySyft	9k	73,462	2	Code Execution
sktime	9k	167,466	3	Code Execution Deserialization Code Injection
optimate	8k	1,029,576	3	Deserialization Deserialization Code Injection
DeepPavlov	7k	16,907	2	Code Execution
guardrails	6k	30,647	1	Code Injection
adversarial-robustness-toolbox	5k	80,632	3	Code Injection
superduper	5k	20,567	1	Code Execution
nexent	4k	54,801	2	Injection
burr	2k	381,872	2	Code Execution
augmentoolkit	1k	5,158,108	1	Code Execution
RAG-Retrieval	1k	3,661,797	3	Deserialization

Table 7: Assigned CVEs

Project	CVEs	Vulnerability Type
optimate	CVE-2026-31217	Code Injection
	CVE-2026-31218	Insecure Deserialization
	CVE-2026-31219	Deserialization
superduper	CVE-2026-31225	Code Execution
TinyZero	CVE-2026-31226	Code Injection
adversarial-robustness-toolbox	CVE-2026-31228	Code Injection
guardrails	CVE-2026-31233	Code Injection
horovod	CVE-2026-31234	Deserialization
imgaug	CVE-2026-31235	Deserialization
llm	CVE-2026-31236	Code Injection
ludwig	CVE-2026-31237	Deserialization
	CVE-2026-31238	Deserialization
mem0	CVE-2026-31240	Incorrect Access Control
	CVE-2026-31241	Incorrect Access Control
	CVE-2026-31242	Incorrect Access Control
gpt-pilot	CVE-2026-31246	OS Command Injection
CosyVoice	CVE-2026-31250	Insecure Deserialization
	CVE-2026-31251	Insecure Deserialization
	CVE-2026-31252	Insecure Deserialization
mamba	CVE-2026-31239	Insecure Deserialization
nexent	CVE-2026-31215	Incorrect Access Control
	CVE-2026-31216	Incorrect Access Control

D Assigned CVE Identifiers

Table 7 shows the CVE identifiers we have gotten.